

Diagrama de arquitectura - Proyecto

Diseño de software

Profesor: Mario Chacón Rivas

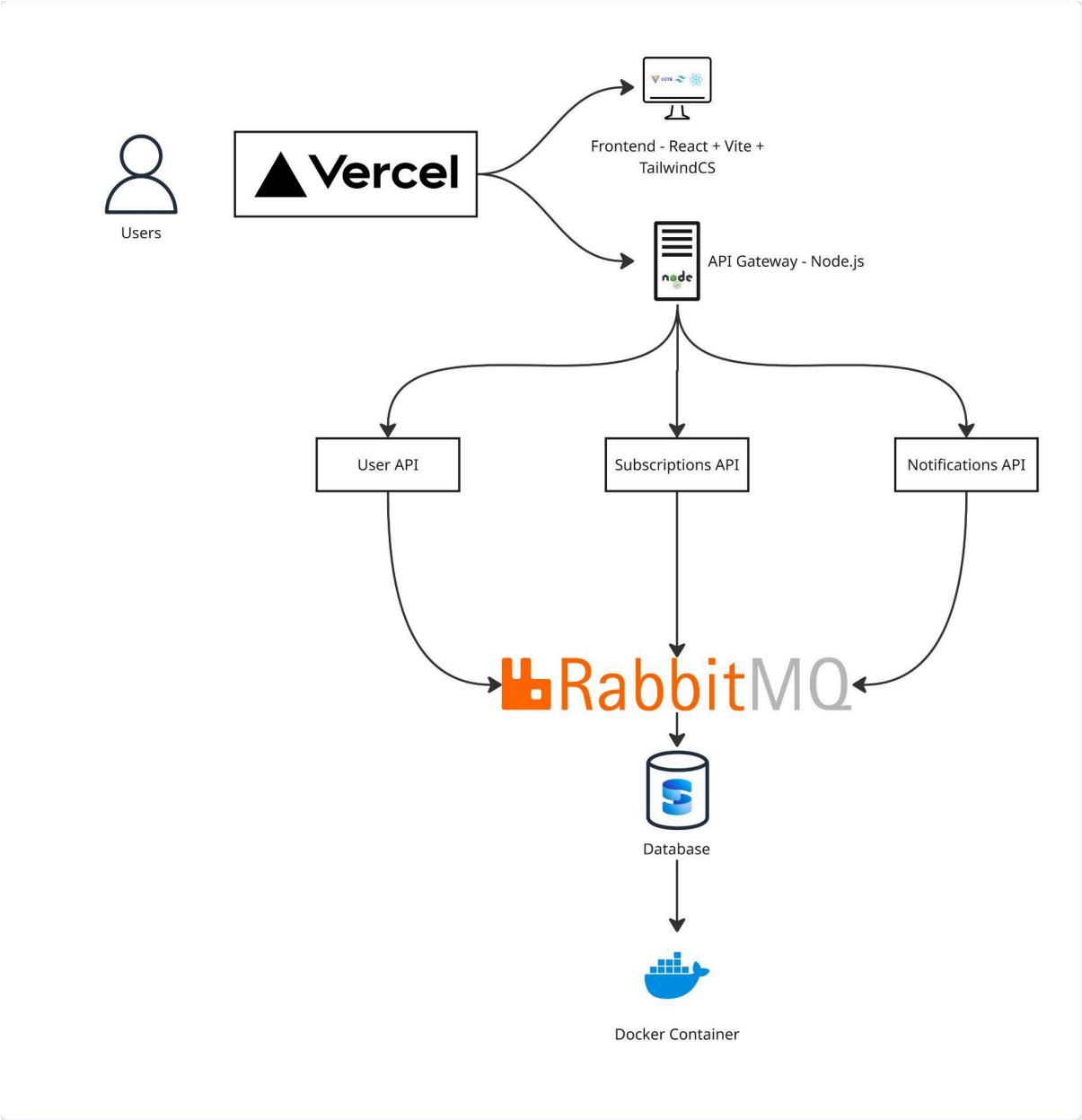
Integrantes:

Sebastian Guillen Guzman

Andrey Jesus Jimenez Nuñez

Alejandro Luna Hernandez

Diagrama:



Explicación con nuestras palabras:

El diagrama consiste en una página web de gestión de suscripciones desplegada en una plataforma en la nube llamada Vercel, esta página está compuesta por un front-end diseñado en React, junto a Vite y Tailwind CSS, además hay un manejo de REST API's donde se usa Node.JS, el cual está apoyado por RabbitMQ cuyo rol es el de recibir y almacenar las notificaciones del sistema.

La base de datos estará en SQL Server el cual va a estar en una imagen de un contenedor de docker para realizar el hosting de forma local, adicional a esto docker también tendrá el rol de ejecutar cada microservicio (User API, Subscriptions API y Notifications API) en su contenedor respectivo.

Explicación de la IA:

Primero, los usuarios interactúan con la aplicación a través de Vercel, que actúa como plataforma de despliegue del frontend. Desde ahí se sirve una aplicación construida con React, Vite y TailwindCSS, lo que sugiere una interfaz rápida, modular y optimizada para rendimiento.

Ese frontend no se comunica directamente con múltiples servicios, sino que centraliza las peticiones en un API Gateway desarrollado en Node.js. Esto es una buena decisión: el gateway funciona como punto único de entrada, manejando autenticación, validación y enrutamiento de solicitudes hacia los servicios internos.

Debajo del API Gateway están los microservicios principales:

- User API: encargado de la gestión de usuarios (registro, autenticación, perfiles, etc.).

- Subscriptions API: maneja la lógica de suscripciones (planes, pagos, estados).
- Notifications API: se encarga de enviar notificaciones (emails, alerts, etc.).

Estos servicios no están completamente acoplados entre sí. En lugar de comunicarse directamente de forma síncrona, utilizan RabbitMQ como sistema de mensajería. Esto permite comunicación asíncrona basada en eventos, lo que mejora la escalabilidad y resiliencia del sistema. Por ejemplo, cuando ocurre una acción en el User API, puede emitir un evento que otros servicios consumen sin necesidad de llamadas directas.

Cada microservicio está diseñado para ejecutarse de forma independiente en su propio contenedor Docker, lo que facilita:

- Despliegue aislado
- Escalado independiente
- Portabilidad entre entornos

Finalmente, todos los servicios interactúan con una base de datos principal, donde se persiste la información del sistema.